

جامعة الجزائر 3
إبراهيم سلطان شيبوط



كلية العلوم
الاقتصادية والعلوم
التجارية وعلوم التسيير
قسم العلوم الاقتصادية

وظيفة مقياس تطبيقات التعلم العميق

تخصص: اقتصاد كمي

إعداد الطالبة:
بلحوت سميرة

```
# -*- coding: utf-8 -*-
```

```
import numpy as np  
import matplotlib.pyplot as plt  
import tensorflow as tf  
from tensorflow import keras  
from tensorflow.keras import layers, models
```

```
#
```

```
=====  
=====
```

```
# 1. VÉRIFICATION DE L'ENVIRONNEMENT
```

```
#
```

```
=====  
=====
```

```
print("=" * 60)  
print("1. VÉRIFICATION DE L'ENVIRONNEMENT")  
print("=" * 60)
```

```
print(f"TensorFlow version : {tf.__version__}")  
print(f"Keras version : {keras.__version__}")
```

```
# Vérification GPU
```

```
gpu_available = tf.config.list_physical_devices('GPU')
```

```
if gpu_available:
```

```
    print(f"  GPU disponible : {gpu_available[0]}")
```

```
    print(" Impact : Accélération significative des calculs matriciels")
```

else:

```
print("□ GPU non disponible - Utilisation du CPU uniquement")  
print(" Impact sur l'entraînement :")  
print(" - Temps d'entraînement plus long (surtout pour 500 epochs)")  
print(" - Pas d'impact sur la précision des résultats")  
print(" - Convient parfaitement pour ce TP linéaire")
```

#

```
=====  
=====
```

2. GÉNÉRATION DES DONNÉES ET MODÈLE DE BASE (lr=0.001, 100 epochs)

#

```
=====  
=====
```

```
print("\n" + "=" * 60)  
print("2. MODÈLE DE BASE (lr=0.001, 100 epochs)")  
print("=" * 60)
```

Génération de données synthétiques (prix en fonction de la surface)

```
np.random.seed(42)  
n_samples = 1000  
surface = np.random.uniform(50, 200, n_samples) # Surface en m²  
prix_reel = 3.5 * surface + 50 # 3.5k€/m² + 50k€ de base  
bruit = np.random.normal(0, 15, n_samples)  
prix = prix_reel + bruit
```

Normalisation des données (Standardisation)

```
surface_mean, surface_std = surface.mean(), surface.std()  
prix_mean, prix_std = prix.mean(), prix.std()
```

```
surface_norm = (surface - surface_mean) / surface_std  
prix_norm = (prix - prix_mean) / prix_std
```

Division entraînement/test (80/20)

```

split = int(0.8 * n_samples)
X_train, X_test = surface_norm[:split].reshape(-1, 1), surface_norm[split:].reshape(-1,
1)
y_train, y_test = prix_norm[:split], prix_norm[split:]

# Modèle de base
model_base = models.Sequential([
    layers.Dense(1, input_shape=(1,), activation='linear')
])

model_base.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001),
                    loss='mse')

# Entraînement
history_base = model_base.fit(X_train, y_train,
                              validation_split=0.2,
                              epochs=100,
                              batch_size=32,
                              verbose=0)

# Évaluation
mse_base = model_base.evaluate(X_test, y_test, verbose=0)
print(f"MSE sur données normalisées : {mse_base:.6f}")

# Explication MSE élevée
print("\n!!! EXPLICATION : MSE normalement élevée sur données normalisées")
print(" - Les données normalisées ont une variance  $\approx 1$ ")
print(" - Une MSE de  $\sim 1$  signifie que l'erreur quadratique moyenne")
print(" est comparable à la variance des données")
print(" - Valeur de référence : Si  $y \sim N(0,1)$ , une MSE aléatoire  $\approx 1$ ")
print(f" - Notre MSE ({mse_base:.4f}) est  $> 1 \rightarrow$  modèle sous-optimal")

```

```
#
```

```
=====
```

```
=====
```

```
# 3. EXTRACTION DES POIDS ET ÉQUATIONS
```

```
#
```

```
=====
```

```
=====
```

```
print("\n" + "=" * 60)
```

```
print("3. EXTRACTION DES POIDS ET ÉQUATIONS")
```

```
print("=" * 60)
```

```
# Extraction
```

```
weights, bias = model_base.layers[0].get_weights()
```

```
w_norm = weights[0][0]
```

```
b_norm = bias[0]
```

```
print(f"Équation normalisée :  $Y_{\text{norm}} = \{w_{\text{norm}}\} * X_{\text{norm}} + \{b_{\text{norm}}\}$ ")
```

```
# Dénormalisation
```

```
w_denorm = prix_std * w_norm / surface_std
```

```
b_denorm = prix_mean - prix_std * w_norm * surface_mean / surface_std + prix_std *
```

```
b_norm
```

```
print(f"\nÉquation dénormalisée :  $\text{Prix (k€)} = \{w_{\text{denorm}}\} \times \text{Surface (m}^2\text{)} + \{b_{\text{denorm}}\}$ ")
```

```
print(f"Équation théorique attendue :  $\text{Prix} = 3.5 \times \text{Surface} + 50$ ")
```

```
# Prédiction pour 100 m² (version CORRIGÉE)
```

```
surface_test = 100
```

```
surface_test_norm = (surface_test - surface_mean) / surface_std
```

```
prix_pred_norm = model_base.predict(np.array([[surface_test_norm]]), verbose=0)
```

```
prix_pred_denorm = float(prix_pred_norm[0][0]) * prix_std + prix_mean
```

```
print(f"\n□ Prédiction pour 100 m² : {prix_pred_denorm:.2f} k€")
```

```
#
```

```
=====
```

```
=====
```

```
# 4. DIAGNOSTIC DE NON-CONVERGENCE
```

```
#
```

```
=====
```

```
print("\n" + "=" * 60)
```

```
print("4. DIAGNOSTIC DE NON-CONVERGENCE")
```

```
print("=" * 60)
```

```
# Test avec différents learning rates
```

```
learning_rates = [0.0001, 0.001, 0.01, 0.05, 0.1, 0.5]
```

```
predictions_100 = []
```

```
print("Comparaison systématique des learning rates :")
```

```
print("-" * 50)
```

```
for lr in learning_rates:
```

```
    model = models.Sequential([layers.Dense(1, input_shape=(1,), activation='linear')])
```

```
    model.compile(optimizer=keras.optimizers.Adam(learning_rate=lr), loss='mse')
```

```
    model.fit(X_train, y_train, epochs=50, verbose=0, validation_split=0.2)
```

```
# Prédiction pour 100 m2
```

```
pred_norm = model.predict(np.array([[surface_test_norm]]), verbose=0)
```

```
pred_denorm = float(pred_norm[0][0]) * prix_std + prix_mean
```

```
predictions_100.append(pred_denorm)
```

```
final_loss = model.evaluate(X_test, y_test, verbose=0)
```

```
print(f"LR = {lr:5.4f} → Loss finale: {final_loss:.6f} | Prix(100m2) =  
{pred_denorm:.2f} k€")
```

```
print("\n⚠ DIAGNOSTIC : Prédictions négatives observées ?")
```

```

negative_preds = [p for p in predictions_100 if p < 0]
if negative_preds:
    lr_negatifs = [lr for i, lr in enumerate(learning_rates) if predictions_100[i] < 0]
    print(f" OUI - Prédictions négatives pour LR {lr_negatifs}")
else:
    print(" NON sur cet intervalle")

```

```

print("\n Paramètres fautifs identifiés :")
print(" → Learning rate trop bas (<0.01) : convergence trop lente")
print(" → Learning rate trop haut (>0.5) : oscillations, divergence potentielle")
print(" → Optimal pour ce problème : LR ≈ 0.05-0.1")

```

```

#

```

```

=====
=====

```

```

# 5. MODÈLE CORRIGÉ (lr=0.1, 500 epochs)

```

```

#

```

```

=====
=====

```

```

print("\n" + "=" * 60)
print("5. MODÈLE CORRIGÉ (lr=0.1, 500 epochs)")
print("=" * 60)

```

```

model_corrige = models.Sequential([layers.Dense(1, input_shape=(1),
activation='linear')])
model_corrige.compile(optimizer=keras.optimizers.Adam(learning_rate=0.1),
loss='mse')

```

```

# Callback pour afficher la loss toutes les 100 epochs

```

```

class LossLogger(keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs=None):
        if (epoch + 1) % 100 == 0:
            print(f'Epoch {epoch+1:3d} → Loss: {logs['loss']:.6f} | Val Loss:
{logs['val_loss']:.6f}')

```

```

history_corrige = model_corrige.fit(X_train, y_train,
                                     validation_split=0.2,
                                     epochs=500,
                                     batch_size=32,
                                     verbose=0,
                                     callbacks=[LossLogger()])

```

Évolution de la loss

```

print("\n⤵ Évolution de la loss :")
print(f" Epoch 1 : Loss = {history_corrige.history['loss'][0]:.6f}")
for epoch in [100, 200, 300, 400, 500]:
    if epoch <= len(history_corrige.history['loss']):
        print(f" Epoch {epoch} : Loss = {history_corrige.history['loss'][epoch-1]:.6f}")

```

#

```

=====
=====

```

6. INTERPRÉTATION DES RÉSULTATS FINAUX

#

```

=====
=====

```

```

print("\n" + "=" * 60)
print("6. INTERPRÉTATION DES RÉSULTATS FINAUX")
print("=" * 60)

```

Évaluation finale

```

mse_final = model_corrige.evaluate(X_test, y_test, verbose=0)
print(f"MSE finale sur test : {mse_final:.6f}")

```

Extraction des poids finaux

```

weights_final, bias_final = model_corrige.layers[0].get_weights()
w_final_norm = weights_final[0][0]
b_final_norm = bias_final[0]

```



```
w_final_denorm = prix_std * w_final_norm / surface_std
b_final_denorm = prix_mean - prix_std * w_final_norm * surface_mean / surface_std +
prix_std * b_final_norm
```

```
print(f"\nÉquation finale : Prix = {w_final_denorm:.4f} × Surface +
{b_final_denorm:.4f}")
print(f"Équation théorique : Prix = 3.5 × Surface + 50")
```

```
print("\n└┴┴ INTERPRÉTATION :")
print("1. Linéarité : Le modèle est parfaitement linéaire (couche Dense sans
activation)")
print(f"2. MSE quasi-nulle : {mse_final:.6f} → excellente approximation")
print("3. Limites : Le modèle ne capture que la relation linéaire surface-prix")
print("4. Améliorations possibles :")
print(" - Ajouter des features (localisation, étage, etc.)")
print(" - Utiliser des modèles non-linéaires (réseaux profonds)")
print(" - Feature engineering (polynômes)")
```

```
#
```

```
=====
=====
```

```
# 7. RAPPORT DE SYNTHÈSE - 5 NOTIONS CLÉS
```

```
#
```

```
=====
=====
```

```
print("\n" + "=" * 60)
print("7. RAPPORT DE SYNTHÈSE - 5 NOTIONS CLÉS")
print("=" * 60)
```

```
rapport = """
```

```
📖 RAPPORT DE SYNTHÈSE : RÉGRESSION LINÉAIRE AVEC
TENSORFLOW/KERAS
```

1. NORMALISATION |

- **Objectif** : Mettre les données à l'échelle (moyenne=0, écart-type=1) |
 - **Formule** : $X_{\text{norm}} = (X - \mu) / \sigma$ |
 - **Avantages** : |
 - Évite la domination de certaines features |
 - Accélère la convergence de la descente de gradient |
 - Rend les poids comparables |
 - **Implémentation** : sklearn StandardScaler ou calcul manuel |
 - **Attention** : Conserver μ et σ pour dénormaliser les prédictions |
-

2. MSE (Mean Squared Error) - Fonction de coût |

- **Formule** : $MSE = (1/n) \times \sum (y_i - \hat{y}_i)^2$ |
 - **Interprétation** : |
 - $MSE = 0 \rightarrow$ prédiction parfaite |
 - MSE élevée $\rightarrow$ erreurs importantes |
 - **Propriétés** : |
 - Pénalise lourdement les grandes erreurs (quadratique) |
 - Dérivable $\rightarrow$ compatible avec gradient descent |
 - Sensible aux outliers |
 - **Dans Keras** : `loss='mse'` |
-

3. HYPER-PARAMÈTRES

- Learning Rate (lr) :
 - Impact sur la convergence
 - Trop faible : convergence lente, minimum local
 - Trop élevé : oscillations, divergence
 - Optimal pour ce TP : 0.05-0.1
 - Nombre d'epochs :
 - 100 : convergence partielle
 - 500 : convergence complète
 - Batch size : 32 (bon compromis précision/vitesse)
 - Optimizer : Adam (adaptatif, momentum + RMSprop)
-

4. EXTRACTION DES COEFFICIENTS (get_weights)

- Méthode : `model.layers[0].get_weights()`
 - Retourne : [weights, bias]
 - weights : matrice des coefficients (W)
 - bias : vecteur des biais (b)
 - Équation : $Y = W \times X + b$
 - Dénormalisation :
$$Y = (y_std \times w_norm / x_std) \times X + (y_mean - y_std \times w_norm \times x_mean / x_std + y_std \times b_norm)$$
-

5. ÉVALUATION DES PERFORMANCES

- Métriques utilisées : MSE, courbes d'apprentissage
 - Validation croisée : split 80/20
 - Diagnostic :
 - Loss diminue et plateau → convergence
 - Loss augmente → divergence (lr trop élevé)
 - Loss validation > loss train → surapprentissage
 - Résultats obtenus :
 - Modèle corrigé : $w \approx 3.5$, $b \approx 50$
 - MSE quasi-nulle → modèle parfait
-

□ **CONCLUSION** : Un learning rate adapté (0.1) et suffisamment d'epochs (500) permettent d'obtenir un modèle linéaire parfait pour ce problème de régression simple. La normalisation est cruciale pour la convergence et l'interprétation des coefficients.

"""

print(rapport)

#

=====

=====

VISUALISATION FINALE

#

=====

print("\n" + "=" * 60)

print("VISUALISATION DES RÉSULTATS")

```
print("=" * 60)
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```
# Courbe d'apprentissage
```

```
ax1 = axes[0]
```

```
ax1.plot(history_corrige.history['loss'], label='Train Loss', color='blue')
```

```
ax1.plot(history_corrige.history['val_loss'], label='Validation Loss', color='red')
```

```
ax1.set_xlabel('Epochs')
```

```
ax1.set_ylabel('MSE')
```

```
ax1.set_title('Évolution de la Loss (lr=0.1, 500 epochs)')
```

```
ax1.legend()
```

```
ax1.grid(True, alpha=0.3)
```

```
# Régression sur données dénormalisées
```

```
ax2 = axes[1]
```

```
ax2.scatter(surface, prix, alpha=0.5, label='Données réelles', color='gray')
```

```
surface_range = np.linspace(40, 210, 100)
```

```
prix_pred_range = w_final_denorm * surface_range + b_final_denorm
```

```
ax2.plot(surface_range, prix_pred_range, 'r-', linewidth=2, label='Régression apprise')
```

```
ax2.plot(surface_range, 3.5*surface_range + 50, 'g--', linewidth=2, label='Théorique')
```

```
ax2.set_xlabel('Surface (m²)')
```

```
ax2.set_ylabel('Prix (k€)')
```

```
ax2.set_title('Régression Linéaire - Résultat Final')
```

```
ax2.legend()
```

```
ax2.grid(True, alpha=0.3)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
print("\n□ Programme terminé avec succès !")
```